Verifying ABA with Leslie

Gaspard Reghem

June 25, 2026

Abstract

Our main objective is the verification of an implementation of Asynchronous Byzantine Agreement. The proof of correctness consist in building a simulation relation between the implementation and an implementation runnning in an idealised setting. The simulation ensures that the correctness of the idealised implementation implies the correctness of the original one. However, this implementation is complex, thus, building the simulation in one go is intractable. Fortunately, the implementation is designed as the interlocking of multiple sub-protocols (GBCA, WCC, RSD, Gather, AVSS, BRB). This architecture allows us to decompose the construction of the overall simulation into smaller steps. Each implementation of sub-protocols will be simulated by an idealised version. Thus, when building the simulation for a protocol using sub-protocols, the sub-implementations will be replaced by their idealised versions; making the construction easier.

1 Preliminaries

1.1 Traditional Framework

Definition 1 (Probabilistic Labelled Transition System)**.** A probabilistic labelled transition system (PLTS) is a tuple $\mathcal{P} := (Q, q_\star, L, T)$ where Q is a (countable) set of states, $q_\star \in Q$ is the initial state, L is a (countable) set of labels and $T \subseteq Q \times (L \cup \{\tau\}) \times \mathcal{D}(Q)$ is a set of probabilistic transitions.

A partial execution of $\mathcal{P}$ is a sequence $q_0.l_0.q_1.l_1 \cdots \in (Q.L)^*.Q \cup (Q.L)^\omega$ such that, for every $n \in \mathbb{N}$, there exists $\mu_n \in \mathcal{D}(Q)$ with $(q_n, l_n, \mu_n) \in T$ and $q_{n+1} \in \bar{\mu_n}$. The set of finite executions is denoted $\mathcal{E}^*$. A partial execution is called an execution if $q_0 = q_\star$. The trace of a partial execution is its projection onto L. The set of traces of $\mathcal{P}$ is denoted $\mathcal{T}$. A scheduler s is a function from $\mathcal{E}^*$ to $\mathcal{D}(T \cup \{\bot\})$ such that, for all $e \in \mathcal{E}^*$ and $t \in s(e)$, $t = \bot$ or $src(t) = last(e)$. The set of schedulers is denoted Sch.

Definition 2 (Partial Probabilistic Execution)**.** Given $\mu \in \mathcal{D}(Q)$ and $s \in Sch$, one can define the partial probabilistic execution induced by (μ, s) as the function $\mathfrak{e}_{\mu,s}$ from $\mathcal{E}^*$ to $[0, 1]$ such that if $e \in Q$ then $\mathfrak{e}_{\mu,s}(e) = \mu(e)$; otherwise, $e = e'.l.q$ and $\mathfrak{e}_{\mu,s}(e) = \mathfrak{e}_{\mu,s}(e') \cdot \sum_{t \in T | lbl(t) = l} s(e')(t) \cdot out(t)(q)$.

A partial probabilistic execution is called a probabilistic execution if it is induced by a pair (μ, s) such that $\mu = \delta_{q_\star}$. If $s \in Sch$ terminates almost surely from μ (i.e. $\sum_{e \in \mathcal{E}^*} \mathfrak{e}_{\mu,s}(e) \cdot s(e)(\bot) = 1$) then $\mathfrak{q}_{\mu,s} = last^{-1} \circ \mathfrak{e}_{\mu,s}$ is a well-defined probability distribution over states. We denote $\mu \overset{l}{\Rightarrow} \mu'$ if and only if there exists $s \in Sch$ such that $\delta_l = \mathfrak{t}_{\mu,s}$ and $\mu' = \mathfrak{q}_{\mu,s}$. Partial probabilistic executions induce a measure on the σ-field induced by cones of partial executions, and the trace function is measurable from the σ-field induced by cones of partial executions to the one induced by cones of traces. Thus, $\mathfrak{t}_{\mu,s} = tr^{-1} \circ \mathfrak{e}_{\mu,s}$ is well-defined and called a trace distribution. The set of trace distributions achievable by $\mathcal{P}$ is defined as those induced by a probabilistic execution of $\mathcal{P}$ and denoted $tdist(\mathcal{P})$. The trace distribution pre-order $\mathcal{P} \subseteq^t \mathcal{P}'$ is defined by $tdist(\mathcal{P}) \subseteq tdist(\mathcal{P}')$.

Definition 3 (Parallel Composition of PLTS)**.** Given two PLTSs $\mathcal{P}$ and $\mathcal{P}'$ and $S \subseteq L \cap L'$, the composed PLTS $\mathcal{P} \parallel_S \mathcal{P}'$ is defined as $(Q \times Q', (q_\star, q'_\star), L \cup L', T \parallel_S T')$ where $((q, q'), l, \mu'') \in T \parallel_S T'$ if and only if

- $l \in S$ and there exist $(q, l, \mu) \in T$ and $(q', l, \mu') \in T'$ such that $\mu \otimes \mu' = \mu''$; or
- $l \notin S$ and there exists $(q, l, \mu) \in T$ such that $\mu \otimes \delta_{q'} = \mu''$; or
- $l \notin S$ and there exists $(q', l, \mu') \in T'$ such that $\delta_q \otimes \mu' = \mu''$.

The trace distribution pre-order is not stable under $\parallel_S$: $\mathcal{P} \subseteq^t \mathcal{P}'$ does not imply $\mathcal{P} \parallel_S \mathcal{P}'' \subseteq^t \mathcal{P}' \parallel_S \mathcal{P}''$ [Seg95]. Therefore, the largest pre-order included in $\subseteq^t$ stable by $\parallel_S$ is considered: $\mathcal{P} \sqsubseteq^t \mathcal{P}'$ if and only if, for all PLTSs $\mathcal{P}''$ and $S \subseteq L \cap L' \cap L''$, $\mathcal{P} \parallel_S \mathcal{P}'' \subseteq^t \mathcal{P}' \parallel_S \mathcal{P}''$.

Lemma 1. *If $\mathcal{P} \sqsubseteq^t \mathcal{P}'$ then, for all PLTSs $\mathcal{P}''$ and $S \subseteq L \cap L' \cap L''$, $\mathcal{P} \parallel_S \mathcal{P}'' \sqsubseteq^t \mathcal{P}' \parallel_S \mathcal{P}''$*

Proof. This is a direct consequence of the associativity of $\parallel_S$. □

Given $R \subseteq A \times \mathcal{D}(B)$, the probabilistic lifting of R, denoted $\mathcal{D}(R)$, is defined by $(\mu_A, \mu_B) \in \mathcal{D}(R)$ if and only if there exists $w : R \to [0, 1]$ such that $\mu_A = a \mapsto \sum_{\mu \in \mathcal{D}(B)} w(a, \mu)$ and $\mu_B = \sum_{(a,\mu) \in R} w(a, \mu) \cdot \mu$.

Definition 4 (Probabilistic Forward Simulation)**.** Given two PLTSs $\mathcal{P}$ and $\mathcal{P}'$, a probabilistic forward simulation of $\mathcal{P}$ by $\mathcal{P}'$ is a relation $\mathcal{S} \subseteq Q \times \mathcal{D}(Q')$ such that $(q_\star, \delta_{q'_\star}) \in \mathcal{S}$ and, for all $(q, \mu') \in \mathcal{S}$ and $(q, l, \mu) \in T$, there exists $\mu' \overset{l}{\Rightarrow} \mu''$ such that $(\mu, \mu'') \in \mathcal{D}(\mathcal{S})$.

Theorem 2 (from [Seg95])**.** *Given two PLTSs $\mathcal{P}$ and $\mathcal{P}'$, $\mathcal{P} \sqsubseteq^t \mathcal{P}'$ if and only if there exists a probabilistic forward simulation of $\mathcal{P}$ by $\mathcal{P}'$.*

Proof. The proof is non-trivial and is detailed in Segala's PhD thesis. Our approach of this proof is detailed in Appendix 3. □

1.2 Non-probabilistic Systems

Non-probabilistic systems can be considered as special instances of PLTSs in which all transitions lead to a Dirac distribution. By identifying each Dirac distribution with the unique state in its support, we may assume that non-probabilistic systems are modelled by LTSs, which leads to several simplifications. In particular, probabilistic forward simulation is equivalent to a simpler simulation relation. On an LTS, we write $q \overset{l}{\Rightarrow} q'$ if there exists a partial execution from q to q' whose trace is l.

Definition 5 (Weak Simulation)**.** Given two LTSs $\mathcal{P}$ and $\mathcal{P}'$, a weak simulation of $\mathcal{P}$ by $\mathcal{P}'$ is a relation $\mathcal{S} \subseteq Q \times Q'$ such that $(q_\star, q'_\star) \in \mathcal{S}$ and, for all $(q, q') \in \mathcal{S}$ and $(q, l, p) \in T$, there exists $q' \overset{l}{\Rightarrow} p'$ such that $(p, p') \in \mathcal{S}$.

Lemma 3. *Given two LTSs $\mathcal{P}$ and $\mathcal{P}'$, $\mathcal{P} \sqsubseteq^t \mathcal{P}'$ if and only if there exists a weak simulation of $\mathcal{P}$ by $\mathcal{P}'$.*

Proof. According to Theorem 2, it suffices to prove that there exists a weak simulation of $\mathcal{P}$ by $\mathcal{P}'$ if and only if there exists a probabilistic forward simulation of $\mathcal{P}$ by $\mathcal{P}'$. For one direction, it suffices to note that any weak simulation between LTSs is a probabilistic forward simulation. For the other direction, if $\mathcal{S}$ is a probabilistic forward simulation of $\mathcal{P}$ by $\mathcal{P}'$ then $\{(q, q') \mid \exists (q, \mu) \in \mathcal{S}, q' \in \bar{\mu}\}$ is a weak simulation of $\mathcal{P}$ by $\mathcal{P}'$. □

There is a natural injection of LTSs into PLTSs, but also a natural projection of PLTSs onto LTSs. It consists in transforming all probabilistic non-determinism into classical non-determinism. The projection of a PLTS $\mathcal{P}$, denoted $\rho(\mathcal{P})$, is the LTS whose transitions are defined by $\{(q, l, q') \mid \exists \mu \in \mathcal{D}(Q), (q, l, \mu) \in T \wedge q' \in \bar{\mu}\}$. ρ is trivially the identity on LTSs, and ρ preserves the set of executions and hence the set of traces. As a result, the notion of weak simulation can be lifted to PLTSs: $\mathcal{S}$ is a weak simulation of $\mathcal{P}$ by $\mathcal{P}'$ if and only if it is a weak simulation of $\rho(\mathcal{P})$ by $\rho(\mathcal{P}')$. Since ρ is a projection onto LTSs, this is consistent with the previous definition.

1.3 Preserving Liveness

Our current definitions are not suited to reasoning about liveness. Indeed, $\mathcal{E}$ always contains the empty execution $q_\star$; thus, $\mathcal{T}$ always contains the empty trace τ. As a result, no liveness property can be satisfied against such a set of executions. However, intuition tells us that such executions should be discarded when evaluating properties, because we implicitly make a progress assumption: "if a transition is enabled then a transition should be performed".

To take this observation into account, we say that an execution is maximal if it is infinite or its last state is a deadlock. Similarly, we say that a trace is maximal if it is the trace of

a maximal execution. According to the progress assumption, properties should be evaluated against the set of maximal traces. Therefore, we must develop simulation relations that preserve the set of maximal traces. The set of maximal traces is denoted $\mathcal{T}^{\max}$.

However, weak simulation does not preserve the set of maximal traces. First, there exists a trivial weak simulation of a deadlock by any LTS. Second, since τ-transitions can be elided, an infinite execution whose trace is τ, called a divergence, can be simulated by stuttering. We call a weak divergence any execution whose trace is τ and which is infinite or ends in a deadlock. We also say that a state weakly diverges if a weak divergence starts from it.

Definition 6 (Maximal Weak Simulation)**.** Given two LTSs $\mathcal{P}$ and $\mathcal{P}'$, a weak simulation $\mathcal{S}$ of $\mathcal{P}$ by $\mathcal{P}'$ is said to be maximal if, for all $(q, q') \in \mathcal{S}$, whenever q weakly diverges, so does q'.

Theorem 4. *Given two PLTSs $\mathcal{P}$ and $\mathcal{P}'$, $\mathcal{T}^{\max} \subseteq \mathcal{T}'^{\max}$ only if there exists a maximal weak simulation of $\mathcal{P}$ by $\mathcal{P}'$.*

Proof. It suffices to adapt the proof of Theorem **??**. When building an execution e' of $\mathcal{P}'$ that simulates a maximal execution e of $\mathcal{P}$, it suffices to check that the additional requirement on the simulation ensures that e' is maximal. If e contains infinitely many non-τ labels then the construction naturally yields an infinite execution. Otherwise, after simulating the smallest finite prefix that contains all non-τ labels, it suffices to trigger the preservation of weak divergence. □

However, this solution is not satisfactory, because the additional requirement asks us to test for the existence of weak divergences in $\mathcal{P}$. In practice, $\mathcal{P}$ will be a PLTS encoding a real implementation whose size and complexity make this search difficult. To circumvent this issue, simulations should rely on tests that can be performed locally (e.g. checking whether a transition is enabled), not globally (e.g. checking whether a divergence exists). We allow global tests on $\mathcal{P}'$ because it will encode a minimal PLTS satisfying a set of properties. We say that a transition is elided by a weak simulation if it is simulated by a stutter.

Definition 7 (Progressive Weak Simulation (inspired by [DSW22]))**.** Given two LTSs $\mathcal{P}$ and $\mathcal{P}'$, a weak simulation $\mathcal{S}$ of $\mathcal{P}$ by $\mathcal{P}'$ is said to be progressive if

- for all $(q, q') \in \mathcal{S}$, if q is a deadlock then q' weakly diverges;
- there exists $<\subseteq Q \times Q$ well-founded such that, for all $(q, q') \in \mathcal{S}$ and $(q, \tau, p) \in T$, if (q, τ, p) is elided by $\mathcal{S}$ then $q > p$ or q' weakly diverges.

Theorem 5. *A weak simulation $\mathcal{S}$ is maximal if and only if it is progressive.*

Proof. Suppose that $\mathcal{S}$ is maximal. Let $(q, q') \in \mathcal{S}$ such that q is a deadlock; then q weakly diverges, so, by maximality, q' weakly diverges. Define $q > p$ if there exists $q' \in Q'$ such that $(q, q') \in \mathcal{S}$, $q \xrightarrow{\tau} p$, $\mathcal{S}$ elides this transition, and q' does not weakly diverge. It remains to check that $>$ is well-founded. If there exists $q_0 > q_1 > \cdots$ then q_0 diverges and there exists $q' \in Q'$ such that $(q_0, q') \in \mathcal{S}$ and q' does not weakly diverge, which contradicts the maximality of $\mathcal{S}$.

Suppose that $\mathcal{S}$ is progressive. Let $(q, q') \in \mathcal{S}$ such that q weakly diverges. If $q \overset{\tau}{\Rightarrow} p$ and p is a deadlock then $q' \overset{\tau}{\Rightarrow} p'$ with $(p, p') \in \mathcal{S}$; thus, p' weakly diverges and so does q'. If q diverges then either q' weakly diverges, or there exists $q' \overset{\tau}{\Rightarrow} p'$ such that p' simulates a suffix of q's divergence by stuttering, contradicting the well-foundedness of $>$. □

Unfortunately, progressive weak simulations are not preserved by composition: if there is a progressive weak simulation of $\mathcal{P}$ by $\mathcal{P}'$, there might not be a progressive weak simulation of $\mathcal{P} \parallel_S$

$\mathcal{P}''$ by $\mathcal{P}' \parallel_S \mathcal{P}''$. Parallel composition might create new deadlocks because of synchronisation. To avoid these issues, we use the IO-automata framework of [LT26].

Definition 8 (Input/Output Automaton)**.** An Input/Output automaton is a PLTS whose set of labels is partitioned into two subsets: input labels In and output labels Out, such that, for all states $q \in Q$ and $in \in In$, there is $\mu \in \mathcal{D}(Q)$ such that $(q, in, \mu) \in T$.

Theorem 6. *Let $\mathcal{P}$, $\mathcal{P}'$ and $\mathcal{P}''$ be IO-automata such that $Out \cap Out'' = \emptyset$. If there exists a progressive weak simulation of $\mathcal{P}$ by $\mathcal{P}'$ then there exists a progressive weak simulation of $\mathcal{P} \parallel_S \mathcal{P}''$ by $\mathcal{P}' \parallel_S \mathcal{P}''$.*

Proof. Let us denote by $\mathcal{S}$ the progressive weak simulation of $\mathcal{P}$ by $\mathcal{P}'$. Then $\mathcal{S}' := \{((q, q''), (q', q'')) \mid (q, q') \in \mathcal{S}\}$ is a weak simulation of $\mathcal{P} \parallel_S \mathcal{P}''$ by $\mathcal{P}' \parallel_S \mathcal{P}''$. To prove that $\mathcal{S}'$ is progressive, it suffices to note that if there is a weak divergence in $\mathcal{P} \parallel_S \mathcal{P}''$ from (q, q'') then q or q'' weakly diverges. This holds because of input-enabledness and the compatibility of IO-automata. It then suffices to apply the maximality of $\mathcal{S}$. □

Since our main simulation relation is probabilistic forward simulation, we need to adapt progressiveness to it. We have seen that probabilistic forward simulation preserves the set of traces because it induces a forward-backward simulation. Thus, we can start by adapting progressiveness to forward-backward simulations. A forward-backward simulation is said to be maximal if, for all $(q, P) \in \mathcal{S}$, whenever q weakly diverges, there exists $p \in P$ that weakly diverges.

Theorem 7. *Given two PLTSs $\mathcal{P}$ and $\mathcal{P}'$, $\mathcal{T}^{\max} \subseteq \mathcal{T}'^{\max}$ if and only if there exists a maximal forward-backward simulation of $\mathcal{P}$ by $\mathcal{P}'$.*

Proof. Based on the construction in the proof of Theorem **??**. □

For the same reason as for weak simulation, testing for weak divergences in $\mathcal{P}$ should be avoided.

Definition 9 (Progressive Forward-Backward Simulation)**.** Given two LTSs $\mathcal{P}$ and $\mathcal{P}'$, a forward-backward simulation $\mathcal{S}$ of $\mathcal{P}$ by $\mathcal{P}'$ is said to be progressive if

- for all $(q, P) \in \mathcal{S}$, if q is a deadlock then there exists $p \in P$ that weakly diverges;
- there exists $<\subseteq Q \times Q$ well-founded such that, for all $(q, P) \in \mathcal{S}$ and $(q, \tau, q') \in T$, $q > q'$ or there exists $p \in P$ that weakly diverges or there exists $P' \subseteq Q'$ such that $(q', P') \in \mathcal{S}$ and, for all $p' \in P'$, there exists $p \in P$ such that $p \overset{\tau}{\Rightarrow} p'$ which is not an elision.

Theorem 8. *A forward-backward simulation $\mathcal{S}$ is maximal if and only if it is progressive.*

Proof. Same proof as for Theorem 5. □

Now, it suffices to modify the definition of probabilistic forward simulation so that the construction of Theorem **??** produces a progressive forward-backward simulation.

Definition 10 (Progressive Probabilistic Forward Simulation)**.** Given two PLTSs $\mathcal{P}$ and $\mathcal{P}'$, a progressive probabilistic forward simulation of $\mathcal{P}$ by $\mathcal{P}'$ is a relation $\mathcal{S} \subseteq Q \times \mathcal{D}(Q')$ such that $(q_\star, \delta_{q'_\star}) \in \mathcal{S}$ and, for all $(q, \mu') \in \mathcal{S}$ and $(q, l, \mu) \in T$, there exists $\mu' \overset{l}{\Rightarrow} \mu''$ such that $(\mu, \mu'') \in \mathcal{D}(\mathcal{S})$. Moreover, if q is a deadlock then there exists $q' \in \bar{\mu}'$ that weakly diverges. Furthermore, there exists $>\subseteq Q \times Q$ well-founded such that, for all $(q, \mu') \in \mathcal{S}$ and $(q, \tau, \mu) \in T$ and $p \in \bar{\mu}$, $q > p$ or there exists $q' \in \bar{\mu}'$ that weakly diverges or there exists $\mu_p \in \mathcal{D}(Q)$ such that $w(p, \mu_p) > 0$ where w is the witness of $(\mu, \mu'') \in \mathcal{S}$ and, for all $q' \in \bar{\mu}' \cap \bar{\mu}_p$, $s(q')(\bot) = 0$ where s is the scheduler s that induces $\mu' \overset{\tau}{\Rightarrow} \mu''$.

Unfortunately, this is not enough to preserve liveness at the level of schedulers. Indeed, the current definition does not preserve maximality of schedulers because it is enough that one state is the probability distribution weakly diverges. To solve that issue, it suffices to replace "there exists $q' \in \bar{\mu}'$ that weakly diverges" by "for all $q' \in \bar{\mu}'$, q' weakly diverges".

1.4 Fairness

Distributed systems suffering from Byzantine failures admit liveness behaviours that are not consistent with the maximality approach. Indeed, Byzantine processes may behave arbitrarily which includes halting, thus, new configurations should be considered as deadlock: states where only Byzantine processes can take a step. Another issue is a monopolisation of the execution: a Byzantine process could make infinitely many arbitrary steps preventing correct processes from making progress. Thus, we need to restrict the set of considered executions.

To do so, we introduce a fairness function $f : T \to \mathbb{B}$ which differentiates transitions taken by correct and Byzantine processes. Then, fair deadlocks are defined as states q where all enabled transitions are unfair and fair divergences are defined as divergences with infinitely many fair transitions. A fair weak divergence is defined as expected and progressive simulations have to be adapted to preserve them. The first step is to replace all mentions of deadlock and divergences by their fair counterpart. Then, the notion of elision has to be revised as simulating a fair internal transition by a weak transiton composed of only unfair transitons should be considered as an elision. Therefore, elision will be fair if the elided transitons is unfair; otherwise, they will be unfair. Finally, the pre-order $>$ has to take into account that difference. In case of an unfair elision, we keep $q > p$, but, in case of fair elision, we allow for $q \geq p$. We need $q \geq p$ because fair divergences include interleavings of fair and unfair transitions.

1.5 Partial Information

We encode partial information by restrictions on the set of schedulers. Intuitively, if the scheduler has not access to all information then it might not be able to distinguish some states or some labels, thus, some distinct executions might look the same to it and it should schedule the same probability distribution of transitions.

Definition 11 (Adversary)**.** An adversary is a couple of observation function $(\mathcal{O}_Q, \mathcal{O}_L)$ which respectively map states and labels to state observation and label observation such that, for all $t, t' \in T$, if $src(t) = src(t')$ and $\mathcal{O}_L(lbl(t)) = \mathcal{O}_L(lbl(t'))$ then $t = t'$.

The injectivity of $\mathcal{O}_L$ on the set of transitions enabled in a given state ensures that the partial-information does not prevent the adversary from resolving non-determinism. An adversary $(\mathcal{O}_Q, \mathcal{O}_L)$ induces an observation $\mathcal{O}_\mathcal{E}$ on executions defined by $\mathcal{O}_\mathcal{E}(q_0.l_0 \cdots) = \mathcal{O}_Q(q_0).\mathcal{O}_L(l_0) \cdots$. An scheduler s is consistent with an adversary if, for all $e, e' \in \mathcal{E}^*$, $\mathcal{O}_\mathcal{E}(e) = \mathcal{O}_\mathcal{E}(e') \implies s(e) = s(e')$. The omniscient adversary is encoded by setting observation functions to identity.

Our simulations are sound for partial-information adversaries because they were designed to be sound for omniscient adversaries. However, there is a completeness gap as partial-information makes more reduction sound. Our chosen solution is to build a PLTS on which omniscient adversary is equivalent to partial-information adversary on the original PLTS. This is the belief construction.

Definition 12 (Belief PLTS)**.** Given a PLTS $\mathcal{P}$ and an adversary $\mathcal{A} := (\mathcal{O}_Q, \mathcal{O}_L)$, the belief

PLTS $\mathcal{P}_{\mathcal{A}}$ is defined as the tuple $(\{\mu \in \mathcal{D}(Q) \mid \mathcal{O}_Q(\bar{\mu}) = \{\cdot\}\}, \delta_{q_\star}, \mathcal{O}_L(L), T_{\mathcal{A}})$ where

$$\mu \xrightarrow{l} \mu' \in T_{\mathcal{A}} \iff \exists (t_q)_{q\in\bar{\mu}} \in T^{\bar{\mu}}, \begin{cases} \forall q \in \bar{\mu}, src(t_q) = q \wedge \mathcal{O}_L(lbl(t_q)) = l \\ \mu' = (\mu_o := q' \mapsto \frac{\mathbb{1}_{\mathcal{O}_Q(q')=o} \cdot \sum_{q\in\bar{\mu}} \mu_q(q')}{\sum_{q,q'\in Q} \mathbb{1}_{\mathcal{O}_Q(q')=o} \cdot \mu_q(q')}) \mapsto \sum_{q,q'\in Q} \mathbb{1}_{\mathcal{O}_Q(q')=o} \cdot \mu_q(q') \end{cases}$$

Theorem 9. *Given a PLTS $\mathcal{P}$ and an adversary $\mathcal{A}$, the set of trace distribution of $\mathcal{P}$ induced by a scheduler consistent with $\mathcal{A}$ is equal to $tdist(\mathcal{P})$.*

An important observation is that the belief construction does not interfere with composite nature of PLTSs. Given $\mathcal{P}$ and $\mathcal{P}'$ two PLTSs and $\mathcal{A}$ and $\mathcal{A}'$ two adversaries (one for each PLTS) and $S \subseteq L \cap L'$, we say that $\mathcal{A}$ and $\mathcal{A}'$ are compatible on S if (i), for all $l \in S$, $\mathcal{O}_L(l) = \mathcal{O}'_L(l)$; (ii) $\mathcal{O}_L(L \setminus S) \cap \mathcal{O}_L(S) = \emptyset$ and (iii) $\mathcal{O}'_L(L' \setminus S) \cap \mathcal{O}'_L(S) = \emptyset$. If $\mathcal{A}$ and $\mathcal{A}'$ are compatible on S, we can define $\mathcal{A} \parallel_S \mathcal{A}'$ as

$$((q, q') \mapsto (\mathcal{O}_Q(q), \mathcal{O}'_Q(q')), l \mapsto \begin{cases} \mathcal{O}_L(l) \text{ if } l \in L \\ \mathcal{O}'_L(l) \text{ if } l \in L' \end{cases})$$

Theorem 10. *Given two PLTSs $\mathcal{P}$ and $\mathcal{P}'$ and their respective adversary $\mathcal{A}$ and $\mathcal{A}'$ and $S \subseteq L \cap L'$, if $\mathcal{A}$ and $\mathcal{A}'$ are compatible on S then $\mathcal{P}_{\mathcal{A}} \parallel_S \mathcal{P}'_{\mathcal{A}'}$ and $(\mathcal{P} \parallel_S \mathcal{P}')_{\mathcal{A}\parallel_S\mathcal{A}'}$ are isomorphic.*

Proof. For clarity, let us denote $\mathcal{P}_1 := \mathcal{P}_{\mathcal{A}} \parallel_S \mathcal{P}'_{\mathcal{A}'}$ and $\mathcal{P}_2 := (\mathcal{P} \parallel_S \mathcal{P}')_{\mathcal{A}\parallel_S\mathcal{A}'}$. First, note that both PLTSs cannot be equal because Q_1 is in $\mathcal{D}(Q) \times \mathcal{D}(Q')$ whereas Q_2 is in $\mathcal{D}(Q \times Q')$. □

2 List of the Protocols

Protocols will be introduced as a set of properties to satisfy given in natural language. Then, an implementation taken from the literature will be presented as pseudo-code. Finally, a PLTS encoding of these properties will be provided to which we aim to reduce the implementation. For readability, pseudo-code and PLTS encoding will be relegated to Appendix.

Common Notation. Without loss of generality, we can assume that each process has a unique identifier ranging from 1 to n. The set of identifier is denoted $[n]$. The number of faulty processes is bounded by a threshold denoted f and the set of faulty processes is denoted F.

2.1 Asynchronous Byzantine Agreement (ABA)

ABA is a canonical protocol where processes must agree on a common output. Processes start with a bit $\{0, 1\}$ and may return a bit. Its interface is composed of a *call* and a *return* statements. A program implements ABA if it satisfies three properties: Validity, Agreement, Almost-Sure Termination.

- Validity: if a correct process returns b then a correct process had b as input.
- Agreement: if two correct processes return, then their outputs are equal.
- Almost-sure Termination: if $n - f$ correct processes take part in the protocol then, with probability 1, all correct processes will eventually return.

Implementation 1 (from [ABDY22])**.** This implementation relies on GBCA and WCC. It proceeds in asynchronous rounds in which processes start by performing GBCA, then call WCC. They use their input as for the first round then update it based on the outcome of GBCA and WCC. The pseudo-code is provided in Algorithm 1.

Definition 13. The PLTS encoding Algorithm 1 is denoted ABA.Core. Its adversary is omniscient.

Specification 1. The PLTS encoding of ABA is provided in Transition System 2. We call it ABA.Spec and its adversary is omniscient. It contains a probabilistic transition whose outcome depends on an unspecified ϵ which corresponds to the goodness of WCC.

Definition 14. ABA.Impl is defined as the parallel composition of ABA.Core, GBCA.Impl and WCC.Impl synchronised on calls, returns and failures. Since the advresaries of ABA.Core and GBCA.Impl are omniscient and the adversary of WCC.Impl is omniscient on its interface, the composed adversary is well-defined.

Definition 15. ABA.Hybrid is defined as the parallel composition of ABA.Core, GBCA.Spec and WCC.Spec synchronised on calls, returns and failures. Since all of their advresaries are omniscient, the composed adversary is well-defined and omniscient.

2.2 ABA Sub-Protocols

Graded Binding Crusader Agreement (GBCA) (taken from [ABDY22, AW23]) Processes start with a binary input $\{0, 1\}$ and may return a value in $\{(0, A), (0, B), (\perp, C), (1, B), (1, A)\}$. A program implements GBCA if it satisfies four properties: Validity, Graded Agreement, Binding, Termination.

- Validity: if a correct process does not return (b, A) then a correct process had $1 - b$ as input.
- Graded Agreement: if two correct processes return (b, X) and (b', X') then $b = 0 \implies b' \neq 1$ and $X = A \implies X' \neq \perp$.
- Binding: when a correct process returns then a bound value b is defined and, in all extensions of the execution, correct processes do not return $(1 - b, B)$ or $(1 - b, A)$.
- Termination: if $n - f$ correct processes take part in the protocol then all correct processes will eventually return.

Implementation 2 (from [ABDY22])**.** This implementation consists of 4 rounds of message exchange. It does not rely on any sub-protocol and its algorithmic core idea is quorum intersection. The pseudo-code can be found in Algorithm 1.

Definition 16. The PLTS encoding Algorithm 1 is denoted GBCA.Impl. Its adversary is omniscient.

Specification 2. The PLTS encoding of GBCA is provided in Transition System 2. It is called GBCA.Spec and its adversary is omniscient. Since the branching property is not preserved by our simulation, we enhance the interface of GBCA to include the bound value in the return labels. This way, the binding property becomes linear.

Weak Common Coin (WCC) Processes start with no input and may return a bit. A program implements WCC if it satisfies three properties: ϵ-Goodness, Unpredictability, Termination.

- ϵ-Goodness: with probability ϵ, all correct processes return 0 (resp. 1).
- Unpredictability: until $f + 1$ processes have called WCC, all outcomes are still possible.

- Termination: if $n - f$ correct processes take part in the protocol then all correct processes will eventually return.

Implementation 3 (simplified from [FKT22])**.** The implementation of WCC relies on SRSD and Gather. The idea is to randomly assign a number to each process then select a subset of processes and return based on the value assigned to these processes.

Definition 17. The PLTS encoding Algorithm 1 is denoted WCC.Core. Its adversary is omniscient.

Specification 3. The PLTS encoding of WCC is provided in Transition System 2. It is called WCC.Spec and its adversary is omniscient. In order to preserve unpredictability, we enhance the interface with a new guess instruction which allows the adversary to make a guess about the future return value. Unpredictability states that this guess cannot be almost-surely correct.

Definition 18. WCC.Impl is defined as the parallel composition of WCC.Core, SRSD.Impl and Gather.Impl synchronised on calls, returns and failures. Since the adversaries of WCC.Core and Gather.Impl are omniscient and the adversary of SRSD.Impl is omniscient on its interface, the composed adversary is well-defined.

Definition 19. WCC.Hybrid is defined as the parallel composition of WCC.Core, Gather.Spec and SRSD.Spec synchronised on calls, returns and failures. Since all of their advresaries are omniscient, the composed adversary is well-defined and omniscient.

2.3 WCC Sub-Protocols

Gather (taken from [SA21, SA24, AFW25]) Processes start with an input $x \in X$ and may return a partial function $f : Proc \rightharpoonup X$ presented as a subset of $Proc \times X$. A program implements Gather if it satisfies four properties: Validity, Agreement, Common Core, Binding, Termination.

- Validity: if a correct process returns f and $f(q)$ is defined and q is correct then $f(q)$ is q's input.
- Agreement: if two correct processes return f and g then $f(q)$ and $g(q)$ are equal or one of them is undefined.
- Binding Common Core: once a correct process has returned, there exists $S \subseteq Proc$ of size at least $n - f$ such that all f returned by a correct process in the future is defined on S.
- Termination: if $n - f$ correct processes take part in Gather then all correct processes will eventually return.

Implementation 4 (taken from [AFW25])**.** This implementation relies on BRB and is non-probabilistic. Basically, each process will broadcast its input and then three rounds of message exchange allow processes to decide which broadcasted inputs they should return.

Definition 20. The PLTS encoding Algorithm 1 is denoted Gather.Core. Its adversary is omniscient.

Specification 4. The PLTS encoding of Gather is provided by Transition System 2. It is denoted Gather.Spec and its adversary is omniscient. Similarly to GBCA, Binding Common Core is transformed into a linear property by enhancing the *return* labels with the bound value.

Definition 21. Gather.Impl is defined as the parallel composition of Gather.Core and BRB.Impl synchronised on calls, returns and failures. Since all of their adversaries of are omniscient, the composed adversary is well-defined and omniscient.

Definition 22. Gather.Hybrid is defined as the parallel composition of Gather.Core and BRB.Spec synchronised on calls, returns and failures. Since all of their advresaries are omniscient, the composed adversary is well-defined and omniscient.

Random Secret Draw (SRSD) The SRSD protocol is actually a pair of protocol: Assignment Asg and Recovery Rec. For each instance, a process is designated beforehand as the owner; all processes know who is the owner. Assignment assigns a random value to the owner such that the assigned value is unknow by all. Recover reveals the assigned value to processes. A correct implementation of SRSD should satisfy five properties.

- Assignment Termination: if all correct processes call SRSD.Asg, then all correct processes will eventually terminate SRSD.Asg.
- Assignment Totality: if a correct process terminates SRSD.Asg then all correct processes will eventually terminate SRSD.Asg.
- Randomness: the return value of SRSD.Rec follows the uniform distribution.
- Secrecy: if no correct process has called SRSD.Rec then the adversary has no information about the return value of SRSD.Rec.
- Retrieval Termination: if all correct processes call SRSD.Rec then all correct processes will eventually terminate SRSD.Rec.

No Agreement properties ???

Implementation 5 (adapted from [FKT22])**.** The implementation relies on AVSS and BRB. Each process randomly pick a number and share it among all processes using AVSS. Then, the owner broadcast a set of processes and all processes reconstruct the secret of these processes and return their sum.

Definition 23. The PLTS encoding Algorithm 1 is denoted SRSD.Core. Its adversary has partial information as it should not see the valuation of `x` when it is picked and when AVSS.Sh is called.

Specification 5. The PLTS encoding of SRSD is provided in Transition System 2. It is called SRSD.Spec and its adversary is omniscient. Similarly to WCC, Secrecy is encoded by adding a guess transition which allows the adversary to predict the outcome of SRSD.Rec. Secrecy is equivalent to the probability that the adversary's prediction is correct being less than $|X|^{-1}$.

Definition 24. SRSD.Impl is defined as the parallel composition of SRSD.Core, BRB.Impl and AVSS.Spec synchronised on calls, returns and failures. The adversary of BRB.Impl is omniscient and the adversary of SRSD.Core is omniscient on its interface with BRB.Impl. Furthermore, the adversary of SRSD.Core and AVSS.Spec agree on their interface. Thus, the composed adversary is well-defined.

Definition 25. SRSD.Hybrid is defined as the parallel composition of SRSD.Core, BRB.Spec and AVSS.Spec synchronised on calls, returns and failures. The adversary of BRB.Impl is omniscient and the adversary of SRSD.Core is omniscient on its interface with BRB.Impl. Furthermore, the adversary of SRSD.Core and AVSS.Spec agree on their interface. Thus, the composed adversary is well-defined.

2.4 Basic Primitives

Byzantine Reliable Broadcast (BRB) Each instance of BRB has a designated *leader* with an input message m and processes may return a received message. A program implements BRB if it satisfies three properties: Validity, Totality and Agreement.

- Validity: if the leader is not corrupted then all correct processes must return its input.
- Agreement: if two correct processes return then they return the same value.
- Totality: if a correct process returns then all correct processes eventually return.

Implementation 6 (taken from [Bra87])**.** This is the classical implementation of BRB composed of three rounds of message exchanges relying on quorum intersection. The pseudo-code is provided in Algorithm 1.

Specification 6. This specification contains linear properties only, thus, the encoding as a PLTS is trivial. It can be found in Transition System 2. Since the leader initiates its instance of BRB, we use only one *call* transition.

Asynchronous Verifiable Secret Sharing (AVSS) The AVSS is actually a pair of protocols: Sharing Sh and Reconstruction Rec. Each instance has a designated process called the dealer which is known by all processes. The dealer starts Sh with a secret value which it divides into shares and hands one to each process. During Rec, processes gather shares from other processes to reconstruct the original secret. A program implements a ϵ-correct AVSS if it satisfies six properties: Completion, Correctness, Secrecy.

- Termination:
  - if the dealer is correct and has called AVSS.Sh then all correct processes will eventually terminate AVSS.Sh;
  - if a correct process has terminated AVSS.Sh then all correct processes will eventually terminate AVSS.Sh;
  - with probability $1 - \epsilon$, if all correct processes have terminated AVSS.Sh and called AVSS.Rec then all correct processes will eventually terminate AVSS.Rec.
- Correctness: Once a correct process has terminated AVSS.Sh, there exists $x \in X$ such that,
  - if the dealer is correct then it has called AVSS.Sh with x;
  - with probability $1 - \epsilon$, no coprevents termination of Shrrect process will terminate AVSS.Rec with a value different from x.
- Secrecy: if the dealer is correct and no correct process has called AVSS.Rec then the adversary has no information about the future retrieved value.

Some properties always hold while others hold with probability $1-\epsilon$. The specification of [CR93] states that all properties hold with probability $1 - \epsilon$, but it seems that the specification can be strengthened. For instance, if the dealer is correct then the commited value can be chosen to be its input to AVSS.Sh. Moreover, the second point of termination should also always hold because correct processes broadcast a certificate to terminate AVSS.Sh, thus, if one correct process gets it then all of them will get it.

Our proof will stop at this level of abstraction, meaning that all considered implementations will be correct up to the correctness of their underlying AVSS. Efficient implementations of AVSS enter the realm of cryptography which is not the main target of our methodology. A possible implementation could have been the one from [CR93].

Specification 7. The PLTS encoding of AVSS is provided in Transition Sytem 2. To encode the commitment as a linear property, the commited value is provided when AVSS.Sh returns. It uses the same trick as WCC and SRSD to encode secrecy. Since secret values are contained in the interface of AVSS, this specification must be expressed with partial information.

3 Proof Decomposition

The end goal is the correctness of ABA.Impl. We establish it via a simulation by ABA.Spec and transport the spec's correctness. The simulation itself is decomposed recursively along the sub-protocol structure.

For a protocol P with concrete sub-protocols $Q_1, \dots, Q_k$, the step "P.Spec simulates P.Impl" is split into:

1. (Substitution) $\mathsf{P.Impl}[\mathsf{Q_1.Spec}, \dots, \mathsf{Q_k.Spec}]$ is simulated by P.Impl, using the child simulations Q_i.Spec simulates Q_i.Impl and compositionality.
2. (Core) P.Spec simulates the hybrid $\mathsf{P.Impl}[\mathsf{Q_1.Spec}, \dots, \mathsf{Q_k.Spec}]$.
3. (Transitivity) Compose (1) and (2).

Leaves of the recursion (no sub-protocol, or only the black-box AVSS.Spec) skip the substitution step.

3.1 Top-level

Theorem 11. *ABA.Concrete satisfies Validity, Agreement and Almost-Sure Termination.*

Proof. According to Theorem 12, ABA.Concrete $\sqsubseteq^f$ ABA.Spec. Moreover, one can prove that ABA.Spec satisfies all three properties which can all be expressed as trace properties or fair trace distribution properties. Thus, by Theorem 2, ABA.Concrete satisfies all three properties. □

3.2 ABA

Theorem 12. *ABA.Concrete* $\sqsubseteq^f$ *ABA.Spec.*

Proof. Trivial by transitivity of $\sqsubseteq^f$ and Lemma 14 and 13. □

Lemma 13. *ABA.Concrete* $\sqsubseteq^f$ *ABA.Hybrid.*

Proof. Both systems only differ by replacing GBCA.Impl (resp. WCC.Concrete) by GBCA.Spec (resp. WCC.Spec). According to Theorem 18 and 15, Lemma 1 implies ABA.Concrete $\sqsubseteq^f$ ABA.Hybrid. □

Lemma 14. *ABA.Hybrid* $\sqsubseteq^f$ *ABA.Spec.*

Proof. Both systems are full-information and probabilistic, thus, according to Theorem 2, it suffices to build a progressive probabilistic forward simulation of ABA.Hybrid by ABA.Spec. [TODO details about the construction] □

3.3 WCC

Theorem 15. *WCC.Concrete* $\sqsubseteq^f$ *WCC.Spec.*

Proof. Trivial by transitivity of $\sqsubseteq^f$ and Lemma 17 and 16. □

Lemma 16. *WCC.Concrete* $\sqsubseteq^f$ *WCC.Hybrid.*

Proof. Both systems only differ by replacing SRSD.Impl (resp. Gather.Impl) by SRSD.Spec (resp. Gather.Spec). According to Theorem 22 and 19, Lemma 1 implies WCC.Concrete $\sqsubseteq^f$ WCC.Hybrid. □

Lemma 17. *WCC.Hybrid* $\sqsubseteq^f$ *WCC.Spec.*

Proof. Both systems are full-information and probabilistic, thus, according to Theorem 2, it suffices to build a progressive probabilistic forward simulation of WCC.Hybrid by WCC.Spec. [TODO details about the construction] □

3.4 GBCA

Theorem 18. *GBCA.Impl* $\sqsubseteq^f$ *GBCA.Spec.*

Proof. Since GBCA is a full-information non-probabilistic protocol, according to Lemma 3, it suffices to build a progressive weak simulation of GBCA.Impl by GBCA.Spec. [TODO details about the construction] □

3.5 Gather

Theorem 19. *Gather.Concrete* $\sqsubseteq^f$ *Gather.Spec.*

Proof. Trivial by transitivity of $\sqsubseteq^f$ and Lemma 21 and 20. □

Lemma 20. *Gather.Concrete* $\sqsubseteq^f$ *Gather.Hybrid.*

Proof. Both systems only differ by replacing BRB.Impl by BRB.Spec. According to Theorem 25, Lemma 1 implies WCC.Concrete $\sqsubseteq^f$ WCC.Hybrid. □

Lemma 21. *Gather.Hybrid* $\sqsubseteq^f$ *Gather.Spec.*

Proof. Since Gather is a full-information non-probabilistic protocol, according to Lemma 3, it suffices to build a progressive weak simulation of Gather.Hybrid by Gather.Spec. [TODO details about the construction] □

3.6 SRSD

Theorem 22. *SRSD.Concrete* $\sqsubseteq^f$ *SRSD.Spec.*

Proof. Trivial by transitivity of $\sqsubseteq^f$ and Lemma 24 and 23. □

Lemma 23. *SRSD.Concrete* $\sqsubseteq^f$ *SRSD.Hybrid.*

Proof. Both systems only differ by replacing BRB.Impl by BRB.Spec. However, since both systems are partial-information, According to Theorem 25, Lemma 1 implies WCC.Concrete $\sqsubseteq^f$ WCC.Hybrid. □

Lemma 24. *SRSD.Hybrid* $\sqsubseteq^f$ *SRSD.Spec.*

Proof. SRSD.Hybrid is partial-information because SRSD.Impl and AVSS.Spec are partial-information. However, SRSD.Spec is full-information. Thus, according to Theorem 9 and 2, it suffices to build a progressive probabilistic forward simulation of the belief PLTS of SRSD.Hybrid by SRSD.Spec. □

3.7 BRB

Theorem 25. *BRB.Impl* $\sqsubseteq^f$ *BRB.Spec.*

Proof. Since BRB is a full-information non-probabilistic protocol, according to Lemma 3, it suffices to build a progressive weak simulation of BRB.Impl by BRB.Spec. [TODO details about the construction] □

Bibliography

[ABDY22] Ittai Abraham, Naama Ben-David, and Sravya Yandamuri. Efficient and adaptively secure asynchronous binary agreement via binding crusader agreement. In *41st ACM Symposium on Principles of Distributed Computing*, pages 381–391, 2022.

[AFW25] Hagit Attiya, Itay Flam, and Jennifer L. Welch. Beyond canonical rounds: Communication abstractions for optimal byzantine resilience. *CoRR*, abs/2510.04310, 2025.

[AW23] Hagit Attiya and Jennifer L. Welch. Multi-valued connected consensus: A new perspective on crusader agreement and adopt-commit. In *27th International Conference on Principles of Distributed Systems (OPODIS)*, 2023.

[Bra87] Gabriel Bracha. Asynchronous byzantine agreement protocols. *Information and Computation*, 75(2):130–143, 1987.

[CR93] Ran Canetti and Tal Rabin. Fast asynchronous byzantine agreement with optimal resilience. In *Proceedings of the Twenty-Fifth Annual ACM Symposium on Theory of Computing*, STOC '93, page 42–51, New York, NY, USA, 1993. Association for Computing Machinery.

[DSW22] Brijesh Dongol, Gerhard Schellhorn, and Heike Wehrheim. Weak Progressive Forward Simulation Is Necessary and Sufficient for Strong Observational Refinement. In Bartek Klin, Sławomir Lasota, and Anca Muscholl, editors, *33rd International Conference on Concurrency Theory (CONCUR 2022)*, volume 243 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 31:1–31:23, Dagstuhl, Germany, 2022. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.

[FKT22] Luciano Freitas, Petr Kuznetsov, and Andrei Tonkikh. Distributed randomness from approximate agreement, 2022.

[LT26] Nancy A Lynch and Mark R Tuttle. An introduction to input/output automata. *Form. Asp. Comput.*, January 2026. Just Accepted.

[SA21] Gilad Stern and Ittai Abraham. Living with asynchrony: the gather protocol. https://decentralizedthoughts.github.io/2021-03-26-living-with-asynchrony-the-gather-protocol, 2021.

[SA24] Gilad Stern and Ittai Abraham. Gather with binding and verifiability. https://decentralizedthoughts.github.io/2024-01-09-gather-with-binding-and-verifiability/, 2024.

[Seg95] Roberto Segala. A compositional trace-based semantics for probabilistic automata. In Insup Lee and Scott A. Smolka, editors, *CONCUR '95: Concurrency Theory*, pages 234–248, Berlin, Heidelberg, 1995. Springer Berlin Heidelberg.

1 Implementation's Pseudo-code

Algorithm 1 Implementation of ABA from [ABDY22]

Input: $\mathtt{b} \in \{0, 1\}$
Sub-Protocols: Countably many instances of GBCA and WCC

$\mathtt{r} \leftarrow 1$
loop
  $(\mathtt{b}, \mathtt{g}) \leftarrow \mathsf{GBCA}_{\mathtt{r}}(\mathtt{b})$ ▷ $\mathtt{b} \in \{0, 1, \perp\}$, $\mathtt{g} \in \{A, B, C\}$
  $\mathtt{c} \leftarrow \mathsf{WCC}_{\mathtt{r}}()$
  if $\mathtt{b} = \perp$:
    $\mathtt{b} \leftarrow \mathtt{c}$
  elif $\mathtt{g} = A$:
    send $\langle \text{DECIDED}, \mathtt{b} \rangle$ to all
  $\mathtt{r} \leftarrow \mathtt{r} + 1$

upon receiving $\langle \text{DECIDED}, \mathtt{b} \rangle$ from $f + 1$ processes and having not sent it:
  send $\langle \text{DECIDED}, \mathtt{b} \rangle$ to all
upon receiving $\langle \text{DECIDED}, \mathtt{b} \rangle$ from $n - f$ processes and having sent it:
  return $\mathtt{b}$

Algorithm 2 Implementation of GBCA from [ABDY22]

Input: $\mathtt{b} \in \{0, 1\}$

send $\langle \text{INPUT}, \mathtt{b} \rangle$ to all processes
upon receiving $\langle \text{INPUT}, \mathtt{b} \rangle$ from at least $f + 1$ processes and not sent $\langle \text{INPUT}, \mathtt{b} \rangle$ yet:
  send $\langle \text{INPUT}, \mathtt{b} \rangle$ to all
upon receiving $\langle \text{INPUT}, \mathtt{b} \rangle$ from at least $n - f$ processes:
  $\mathtt{Valid} \leftarrow \mathtt{Valid} \cup \{\mathtt{b}\}$
  if not sent $\langle \text{ECHO}, \cdot \rangle$ yet **then** send $\langle \text{ECHO}, \mathtt{b} \rangle$ to all
wait until
  (a) receiving $\langle \text{ECHO}, \mathtt{b} \rangle$ from at least $n - f$ processes **or**
  (b) receiving $\langle \text{ECHO}, \cdot \rangle$ from at least $n - f$ processes and $|\mathtt{Valid}| > 1$
  if (a) **then** send $\langle \text{VOTE}, \mathtt{b} \rangle$ to all **else** send $\langle \text{VOTE}, \perp \rangle$ to all
wait until
  (a) receiving $\langle \text{VOTE}, \mathtt{b} \rangle$ from at least $n - f$ processes **or**
  (b) receiving $\langle \text{VOTE}, \cdot \rangle$ from at least $n - f$ processes and $|\mathtt{Valid}| > 1$
  if (a) **then** send $\langle \text{BIND}, \mathtt{b} \rangle$ to all **else** send $\langle \text{BIND}, \perp \rangle$ to all
wait until
  (a) receiving $\langle \text{BIND}, \mathtt{b} \rangle$ from at least $n - f$ processes **or**
  (b) receiving $\langle \text{BIND}, \cdot \rangle$ from at least $n - f$ processes and
  there exists $\mathtt{b} \in \{0, 1\}$ such that $\langle \text{BIND}, \mathtt{b} \rangle$ has been received once and
  $\langle \text{VOTE}, \mathtt{b} \rangle$ has been received from $f + 1$ processes and $|\mathtt{Valid}| > 1$ **or**
  (c) receiving $\langle \text{BIND}, \perp \rangle$ from at least $n - f$ processes and $|\mathtt{Valid}| > 1$
  if (a) **then** return $(\mathtt{b}, A)$ **elif** (b) **then** return $(\mathtt{b}, B)$ **else** return $(\perp, C)$

Algorithm 3 Implementation of WCC for process p_{id} adapted from [FKT22]

Input:
Sub-Protocols: 1 instance of Gather, n instances of SRSD (process p_k is the owner of SRSD_k)

$\forall k \in [n], \mathsf{SRSD}_k.\mathsf{Asg}.call()$
wait until $\mathsf{SRSD}_{id}.\mathsf{Asg}.return()$
  $\mathsf{Gather}.call(id)$
upon $\mathsf{Gather}.return(S)$
  $\forall k \in S$, $\mathsf{SRSD}_k.\mathsf{Rec}.call()$
  $\mathtt{T} \leftarrow \emptyset$
upon $\mathsf{SRSD}_k.\mathsf{Rec}.return(x)$ and $k \in S$
  $\mathtt{T} \leftarrow \mathtt{T} \cup \{x\}$
upon $|\mathtt{T}| = |S|$
  return 0 if $0 \in \mathtt{T}$ else 1

Algorithm 4 Implementation of Gather from [AFW25]

Input: $\mathtt{m}$
Sub-Protocols: n instances of BRB (p_k is leader in BRB_k)

$\mathsf{BRB}_{id}.call(\mathtt{m})$
upon $\mathsf{BRB}_k.return(\mathtt{m'})$:
  $\mathtt{AP} \leftarrow \mathtt{AP} \cup \{(k, \mathtt{m'})\}$
upon $|\mathtt{AP}| \geq n - f$ and not sent $\langle \text{ECHO}, \cdot \rangle$:
  send $\langle \text{ECHO}, \mathtt{AP} \rangle$ to all
upon receiving approved $\langle \text{ECHO}, \mathtt{AP}_{id} \rangle$ from at least $n - f$ processes and not sent $\langle \text{VOTE}, \cdot \rangle$:
  send $\langle \text{VOTE}, \bigcup_{id} \mathtt{AP}_{id} \rangle$ to all
upon receiving approved $\langle \text{VOTE}, \mathtt{AP}_{id} \rangle$ from at least $n - f$ processes and not sent $\langle \text{BIND}, \cdot \rangle$:
  send $\langle \text{BIND}, \bigcup_{id} \mathtt{AP}_{id} \rangle$ to all
upon receiving approved $\langle \text{BIND}, \mathtt{AP}_{id} \rangle$ from at least $n - f$ processes:
  return $\bigcup_{id} \mathtt{AP}_{id}$

Algorithm 5 Implementation of SRSD for process p_{id} adapted from [FKT22]

Input:
Sub-Protocols: 1 instances of BRB where p_o is leader and n instances of AVSS (process p_k is dealer in AVSS_k)

Protocol: SRSD.Asg()
$\mathtt{S}, \mathtt{x} \leftarrow \emptyset, \mu$ ▷ μ: uniform distribution over the domain of secrets
$\mathsf{AVSS}_{id}.\mathsf{Sh}.call(\mathtt{x})$
upon $\mathsf{AVSS}_k.\mathsf{Sh}.return()$ and p_{id} is the owner:
  $\mathtt{S} \leftarrow \mathtt{S} \cup \{k\}$
upon $|\mathtt{S}| \geq n - f$ and p_{id} is the owner:
  $\mathsf{BRB}.call(S)$
upon $\mathsf{BRB}.return(S')$ and $\forall k \in S', \mathsf{AVSS}_k.\mathsf{Sh}.return()$:
  $\mathsf{src} \leftarrow S'$
  return

Protocol: SRSD.Rec()
$\mathtt{sum} \leftarrow 0$
$\forall k \in \mathtt{src}, \mathsf{AVSS}_k.\mathsf{Rec}.call()$
upon $\mathsf{AVSS}_k.\mathsf{Rec}.return(x)$ and $k \in \mathtt{src}$:
  $\mathtt{sum} \leftarrow \mathtt{sum} + x$
wait until all $\mathsf{AVSS}_k.\mathsf{Rec}$ have returned for $k \in \mathtt{src}$:
  return $\mathtt{sum}$

Algorithm 6 Implementation of BRB for process p_{id} from [Bra87]

Input: `m`

if p_{id} is leader:
  send $\langle \text{INIT}, \texttt{m} \rangle$ to all
upon receiving $\langle \text{INIT}, \texttt{m} \rangle$ from leader and having not sent ECHO message:
  send $\langle \text{ECHO}, \texttt{m} \rangle$ to all
upon receiving $\langle \text{ECHO}, \texttt{m} \rangle$ from $n - f$ processes and having not sent VOTE message:
  send $\langle \text{VOTE}, \texttt{m} \rangle$ to all
upon receiving $\langle \text{VOTE}, \texttt{m} \rangle$ from $f + 1$ processes and having not sent VOTE message:
  send $\langle \text{VOTE}, \texttt{m} \rangle$ to all
upon receiving $\langle \text{VOTE}, \texttt{m} \rangle$ from $n - f$ processes:
  return `m`

2 Specification

Transition System 1 Encoding of ABA

State: $call \in \{0,1,\bot\}^{[n]}; ret \in \mathbb{B}^{[n]}; bind, val \in \{0,1,\bot\}; coin \in \{0,1,\bot,\top\}; F \subseteq [n]$
Label: $\{\text{call}(id,b), \text{return}(id,b), \text{fail}(id) \mid id \in [n], b \in \mathbb{B}\}$
Input: $\{\text{call}(id,b), \text{fail}(id) \mid id \in [n], b \in \mathbb{B}\}$
Fairness: all transitions are fair except those labelled by fail and the call loops
Initial: $call = \bot^{[n]}; ret = \bot^{[n]}; bind = \bot; out = \bot; val = \bot; F = \emptyset$

$call[id] = \bot \wedge bind = \bot \xrightarrow{\text{call}(id,b)} call[id] = b$
$\top \xrightarrow{\text{call}(id,b)} \top$ ▷ loop for input-enabledness
$|\{id \in [n] \setminus F \mid call[id] \neq \bot\} \cup F| \geq n - f \wedge \forall id \in [n] \setminus F, call[id] \neq b \xrightarrow{\tau} call = \bot^{[n]} \wedge bind = 1 - b \wedge val = 1 - b \wedge coin = \bot$
$|\{id \in [n] \setminus F \mid call[id] \neq \bot\} \cup F| \geq n - f \wedge \exists id, id' \in [n] \setminus F, call[id] = 1 \wedge call[id'] = 0 \xrightarrow{\tau} call = \bot^{[n]} \wedge bind = 1 - b \wedge coin = \bot$
$call = \bot^{[n]} \wedge bind \neq \bot \xrightarrow{\tau} coin = 1$ with probability ϵ; 0 with probability ϵ; $\top$ otherwise
$call[id] = \bot \wedge bind = coin \xrightarrow{\tau} call[id] = bind$
$call[id] = \bot \wedge bind \neq coin \xrightarrow{\tau} call[id] = b$ ▷ $b \in \mathbb{B}$
$val = b \wedge ret[id] = \bot \xrightarrow{\text{return}(id,b)} ret[id] = \top$
$id \notin F \wedge |F| < f \xrightarrow{\text{fail}(id)} F = F \cup \{id\}$
$\top \xrightarrow{\text{fail}(id)} \top$ ▷ loop for input-enabledness

Transition System 2 Encoding of GBCA

State: $call \in \{0,1,\bot\}^{[n]}; ret \in \mathbb{B}^{[n]}; bind, grade \in \{0,1,\bot\}; F \subseteq [n]$
Label: $\{\text{call}(id,b), \text{return}(id,x,b), \text{fail}(id) \mid id \in [n], x \in (\mathbb{B} \times \{A,B\}) \cup \{(\bot,C)\}, b \in \mathbb{B}\}$
Input: $\{\text{call}(id,b), \text{fail}(id) \mid id \in [n], b \in \mathbb{B}\}$
Fairness: all transitions are fair except those labelled by fail and the call loops
Initial: $call = \bot^{[n]}; ret = \bot^{[n]}; bind = \bot; grade = \bot; F = \emptyset$

$call[id] = \bot \xrightarrow{\text{call}(id,b)} call[id] = b$
$\top \xrightarrow{\text{call}(id,b)} \top$ $\triangleright$ loop for input-enabledness
$|\{id \in [n] \setminus F \mid call[id] \neq \bot\} \cup F| \geq n - f \wedge \exists id \in [n] \setminus F, call[id] = b \wedge bind = \bot \xrightarrow{\tau} bind = b$
$bind \neq \bot \wedge \exists id \in [n] \setminus F, call[id] = 1 - bind \wedge ret[id] = \bot \xrightarrow{\text{return}(id,(bind,B),bind)} ret[id] = \top$
$bind \neq \bot \wedge grade \in \{\bot, 1\} \wedge ret[id] = \bot \xrightarrow{\text{return(id,(bind,A),bind)}} grade = 1 \wedge ret[id] = \top$
$bind \neq \bot \wedge \exists id \in [n] \setminus F, call[id] = 1 - bind \wedge grade \in \{\bot, 0\} \wedge ret[id] = \bot \xrightarrow{\text{return}(id,(\bot,C),bind)}$
$grade = 0 \wedge ret[id] = \top$
$id \notin F \wedge |F| < f \xrightarrow{\text{fail}(id)} F = F \cup \{id\}$
$\top \xrightarrow{\text{fail}(id)} \top$ $\triangleright$ loop for input-enabledness

Transition System 3 Encoding of WCC

State: $call, ret \in \mathbb{B}^{[n]}; val \in \{0,1,\bot,\top\}; guess \in \{0,1,\bot\}, F \subseteq [n]$
Label: $\{\text{call}(id), \text{return}(id,b), \text{fail}(id), \text{guess}(b) \mid id \in [n], b \in \mathbb{B}\}$
Input: $\{\text{call}(id), \text{fail}(id) \mid id \in [n]\}$
Fairness: all transitions are fair except those labelled by fail and the call loops
Initial: $call = \bot^{[n]}; ret = \bot^{[n]}; val = \bot; F = \emptyset$

$call[id] = \bot \xrightarrow{\text{call}(id)} call[id] = \top$
$\top \xrightarrow{\text{call}(id)} \top$ $\triangleright$ loop for input-enabledness
$|\{id \in [n] \setminus F \mid call[id] \neq \bot\} \cup F| > f \wedge val = \bot \xrightarrow{\tau} val = 0$ with probability ϵ; 1 with probability ϵ; $\top$ otherwise
$val \in \{b, \top\} \wedge ret[id] = \bot \xrightarrow{\text{return}(id,b)} ret[id] = \top$ $\triangleright\ b \in \mathbb{B}$
$id \notin F \wedge |F| < f \xrightarrow{\text{fail}(id)} F = F \cup \{id\}$
$\top \xrightarrow{\text{fail}(id)} \top$ $\triangleright$ loop for input-enabledness
$val = \bot \wedge guess = \bot \xrightarrow{\text{guess}(b)} guess = b$

Transition System 4 Encoding of Gather

State: $call \in (X \cup \{\bot\})^{[n]}; ret \in \mathbb{B}^{[n]}; bind, F \subseteq [n]$
Label: $\{\text{call}(id, x), \text{return}(id, g, bind), \text{fail}(id) \mid id \in [n], x \in X, g \in [n] \rightharpoonup X, bind \subseteq [n]\}$
Input: $\{\text{call}(id, x), \text{fail}(id) \mid id \in [n], x \in X\}$
Fairness: all transitions are fair except those labelled by fail and the call loops
Initial: $call = \bot^{[n]}; ret = \bot^{[n]}; bind = \emptyset; F = \emptyset$

$call[id] = \bot \xrightarrow{\text{call}(id,x)} call[id] = x$
$\top \xrightarrow{\text{call}(id,x)} \top$ ▷ loop for input-enabledness
$call[id] = \bot \wedge id \in F \xrightarrow{\tau} call[id] = x$
$|\{id \in [n] \mid call[id] \neq \bot\}| > n - f \wedge bind = \emptyset \wedge S \subseteq \{id \in [n] \mid call[id] \neq \bot\} \xrightarrow{\tau} bind = S$
$bind \neq \bot \wedge ret[id] = \bot \wedge g$ is defined on $bind \wedge g$ agrees with $call \xrightarrow{\text{return}(id,g,bind)} ret[id] = \top$
$id \notin F \wedge |F| < f \xrightarrow{\text{fail}(id)} F = F \cup \{id\}$
$\top \xrightarrow{\text{fail}(id)} \top$ ▷ loop for input-enabledness

Transition System 5 Encoding of SRSD

State: $Asg.call, Asg.ret, Rec.call, Rec.ret \in \mathbb{B}^{[n]}; val, guess \in X \cup \{\bot\}; F \subseteq [n]$
Label: $\{\text{Asg.call}(id), \text{Asg.return}(id), \text{Rec.call}(id), \text{Rec.return}(id, x), \text{fail}(id), guess(x) \mid id \in [n], x \in X\}$
Input: $\{\text{Asg.call}(id), \text{Rec.call}(id), \text{fail}(id) \mid id \in [n]\}$
Fairness: all transitions are fair except those labelled by fail and the call loops
Initial: $Asg.call, Asg.ret, Rec.call, Rec.ret = \bot^{[n]}; val, guess = \bot; F = \emptyset$

$Asg.call[id] = \bot \xrightarrow{\text{Asg.call}(id)} Asg.call[id] = \top$
$\top \xrightarrow{\text{Asg.call}(id)} \top$ ▷ loop for input-enabledness
$|\{id \in [n] \setminus F \mid call[id] \neq \bot\} \cup F| \geq n - f \wedge Asg.ret[id] = \bot \xrightarrow{\text{Asg.return}(id)} Asg.ret[id] = \top$
$Rec.call[id] = \bot \xrightarrow{\text{Rec.call}(id)} Rec.call[id] = \top$
$\top \xrightarrow{\text{Rec.call}(id)} \top$ ▷ loop for input-enabledness
$|\{id \in [n] \setminus F \mid call[id] \neq \bot\} \cup F| \geq n - f \wedge val = \bot \xrightarrow{\tau} val = x$ with probability $|X|^{-1}$
$val \neq \bot \wedge Rec.ret[id] = \bot \xrightarrow{Rec.return(id,val)} Rec.ret[id] = \top$
$id \notin F \wedge |F| < f \xrightarrow{\text{fail}(id)} F = F \cup \{id\}$
$\top \xrightarrow{\text{fail}(id)} \top$ ▷ loop for input-enabledness
$\forall id \in [n] \setminus F, Rec.call[id] = \bot \wedge guess = \bot \xrightarrow{\text{guess}(x)} guess = x$

Transition System 6 Encoding of BRB

State: $call \in M \cup \mathbb{B}; ret \in \mathbb{B}^{[n]}; F \subseteq [n]$
Label: $\{\text{call}(m), \text{return}(id, x), \text{fail}(id) \mid id \in [n], m \in M\}$
Input: $\{\text{call}(m), \text{fail}(id) \mid id \in [n], m \in M\}$
Fairness: all transitions are fair except those labelled by fail and the call loops
Initial: $call = \bot; ret = \bot^{[n]}; F = \emptyset$

$call = \bot \xrightarrow{\text{call}(m)} call = m$
$\top \xrightarrow{\text{call}(id,b)} \top$ ▷ loop for input-enabledness
leader is in $F \wedge ret = \bot^{[n]} \xrightarrow{\tau} call = \top$
$call \in M \wedge ret[id] = \bot \xrightarrow{\text{return}(id,call)} ret[id] = \top$
$id \notin F \wedge |F| < f \xrightarrow{\text{fail}(id)} F = F \cup \{id\}$
$\top \xrightarrow{\text{fail}(id)} \top$ ▷ loop for input-enabledness

Transition System 7 Encoding of AVSS

State: $Sh.call, Rec.ret \in X \cup \mathbb{B}; Sh.ret, Rec.call \in \mathbb{B}^{[n]}; guess \in X \cup \{\bot\}; F \subseteq [n]$
Label: $\{\text{Sh.call}(x), \text{Sh.return}(id, x), \text{Rec.call}(id), \text{Rec.return}(id, x), \text{fail}(id), guess(x) \mid id \in [n], x \in X\}$
Input: $\{\text{Sh.call}(id), \text{Rec.call}(id), \text{fail}(id) \mid id \in [n]\}$
Fairness: all transitions are fair except those labelled by fail and the call loops
Initial: $Sh.call = \bot; Sh.ret, Rec.call, Rec.ret = \bot^{[n]}; val, guess = \bot; F = \emptyset$
Partial Information: $\mathcal{O}_Q$ erases the valuation of $Sh.call$ if it is in X and $\mathcal{O}_L$ removes x in $Sh.call(x)$

$Sh.call = \bot \xrightarrow{\text{Sh.call}(x)} Sh.call = x$
$\top \xrightarrow{\text{Sh.call}(x)} \top$ ▷ loop for input-enabledness
dealer is in $F \wedge Sh.ret = \bot^{[n]} \xrightarrow{\tau} Sh.call = \top$ ▷ Bad dealer prevents termination of Sh
dealer is in $F \wedge Sh.ret = \bot^{[n]} \xrightarrow{\tau} Sh.call = x$ ▷ Bad dealer commits an arbitrary value
$Sh.call \in X \wedge ret[id] = \bot \xrightarrow{Sh.return(id,Sh.call)} ret[id] = \top$
$Rec.call[id] = \bot \xrightarrow{\text{Rec.call}(id)} Rec.call[id] = \top$
$\top \xrightarrow{\text{Rec.call}(id)} \top$ ▷ loop for input-enabledness
$|\{id \in [n] \setminus F \mid Rec.call[id] \neq \bot\} \cup F| \geq n - f \wedge Rec.ret[id] = \bot \xrightarrow{\tau} Rec.ret[id] = \top$ with probability $\epsilon; x \in X$ with probability $(1-\epsilon) * \epsilon; Sh.call$ otherwise
$Rec.ret[id] \in X \xrightarrow{Rec.return(id,Rec.ret[id])} Rec.ret[id] = \top$
$id \notin F \wedge |F| < f \xrightarrow{\text{fail}(id)} F = F \cup \{id\}$
$\top \xrightarrow{\text{fail}(id)} \top$ ▷ loop for input-enabledness
$\forall id \in [n] \setminus F, Rec.call[id] = \bot \wedge guess = \bot \xrightarrow{\text{guess}(x)} guess = x$

3 Proof of Theorem 2

In our setting, the most interesting implication of Theorem 2 is soundness. We will see that the definition of $\sqsubseteq^t$ has some limitations, notably with respect to liveness. To address this, we introduce a stronger notion of trace distribution and strengthen probabilistic forward simulation accordingly. However, the soundness proof of Segala [Seg95] works at the level of $\mathfrak{e}_{\mu,s}$, not s. This is problematic because our strengthening consists in constraining schedulers; thus, we need to adapt the proof. If $\mathcal{S}$ is a probabilistic forward simulation then it induces a choice function $\mathcal{C} : \mathcal{S} \times T \to Sch \times (\mathcal{S} \to [0,1])$ which maps any $(q, \mu') \in \mathcal{S}$ and $(q, l, \mu) \in T$ to the scheduler that induces $\mu' \overset{l}{\Rightarrow} \mu''$ and the witness w of $(\mu, \mu'') \in \mathcal{D}(\mathcal{S})$.

Algorithm 7 Simulation Algorithm for Schedulers

Input: $\mathcal{P}, \mathcal{P}'$ two PLTSs, $\mathcal{S}$ a probabilistic forward simulation of $\mathcal{P}$ by $\mathcal{P}'$ and $s \in Sch$

$\mathtt{e}, \mathtt{e'}, \mathtt{mu'} \leftarrow q_\star, q'_\star, \delta_{q'_\star}$
$\mathtt{acc} \leftarrow \mathtt{e'}$
$\mathtt{t} \leftarrow s(\mathtt{e})$
while $\mathtt{t} \neq \bot$ **do**
  $\mathtt{s'}, \mathtt{w} \leftarrow \mathcal{C}(last(e), \mu', t)$
  $\mathtt{t'} \leftarrow \mathtt{s'}(\mathtt{e'})$
  while $\mathtt{t'} \neq \bot$ **do**
    $\mathtt{e'} \leftarrow \mathtt{e'}.lbl(\mathtt{t'}).out(\mathtt{t'})$
    $\mathtt{acc} \leftarrow \mathtt{acc}.lbl(\mathtt{t'}).out(\mathtt{t'})$
    $\mathtt{t'} \leftarrow \mathtt{s'}(\mathtt{e'})$
  end while
  $\mathtt{e'}, \mathtt{mu'} \leftarrow last(\mathtt{e'}), \mathfrak{q}_{\mathtt{mu'},\mathtt{s'}}$
  $\mathtt{e} \leftarrow \mathtt{e}.lbl(\mathtt{t}).q$ with probability $\frac{\sum_{\mu \in \mathcal{D}(Q')} w(q,\mu) \cdot \mu(\mathtt{e'})}{\mathtt{mu'}(\mathtt{e'})}$
  $\mathtt{t} \leftarrow \mathtt{s}(\mathtt{e})$
end while

Once the choice function $\mathcal{C}$ is fixed, Algorithm 7 is a sequential probabilistic program; thus, it can be modelled as a deterministic PLTS. Of course, the validity of the algorithm's individual instructions should be established rigorously, but this is not our concern here. The variable $\mathtt{e}$ encodes an execution of $\mathcal{P}$ and $\mathtt{acc}$ an execution of $\mathcal{P}'$ that simulates it. In order to extract a scheduler of $\mathcal{P}'$ from Algorithm 7, we need to identify the instructions deciding which transitions to execute in $\mathcal{P}'$. The most obvious one is $\mathtt{t'} \leftarrow \mathtt{s'}(\mathtt{e'})$ when $\mathtt{t'} \neq \bot$; then there is $\mathtt{t} \leftarrow \mathtt{s}(\mathtt{e})$ when $\mathtt{t} = \bot$. The case where $\mathtt{t'} = \bot$ has to be discarded because another transition will be picked without updating $\mathtt{acc}$. When $\mathtt{t} = \bot$, the algorithm stops, so the scheduler on $\mathcal{P}'$ also returns $\bot$. After each update of $\mathtt{acc}$, one of the three following events occurs: (i) $\mathtt{t}$ is set to $\bot$; (ii) $\mathtt{t'}$ is set to a non-$\bot$ value; or (iii) $\mathtt{t'}$ is set to $\bot$ indefinitely. Cases (i) and (iii) correspond to scheduling $\bot$ and case (ii) corresponds to scheduling $\mathtt{t'}$. Thus, the key values to compute are $reach(e')$, the probability of reaching a configuration where $\mathtt{acc} = e'$, and $sch(e', t)$, the probability of reaching a configuration where $\mathtt{acc} = e$ and $\mathtt{t'} = t$. Knowing these, one can define a scheduler on $\mathcal{P}'$ as

$$s' : e' \mapsto \begin{cases} t' \mapsto \frac{sch(e',t')}{reach(e')} \\ \bot \mapsto 1 - \frac{\sum_{t' \in T'} sch(e',t')}{reach(e')} \end{cases}$$

We conjecture that $\mathfrak{t}_{q_\star,s} = \mathfrak{t}_{q'_\star,s'}$.